SHADER PROGRAM COMPILER: 2D/3D GRAPHICS & GAME DSL ENGINE

A High-Level Declarative Domain-Specific Language (DSL) & C++ Source-to-Source Compiler

Name	Eldho
Reg. No	24BAI0268
Course	Compiler Design (Semester 3)
Official Repository	https://github.com/eldhoGI2/shader-compiler

Team Members: Hari (24BAI0262), Soujanya (24BCI0135), Pragadeeswar (24BCE0618)

1. Executive Summary & Project Objectives (What We Have Done)

Modern computer graphics programming in languages such as C and C++ requires managing extensive low-level boilerplate: window creation, pixel format configuration, memory allocation, OpenGL context lifecycle management, raw pointer handling, and matrix mathematics. For developers, students, and graphics designers, this steep learning curve obscures the creative process and introduces frequent state-machine bugs, context leaks, and segmentation faults.

The primary goal of this project is to build an end-to-end Domain-Specific Language (DSL) and Source-to-Source Compiler (Transpiler) that enables users to author rich 2D vector art, compile hardware-accelerated GPU fragment shaders, play interactive 2D arcade games with collision physics, and navigate 3D first-person environments using clean, human-readable, declarative syntax.

The compiler takes custom declarative '.my' scripts and translates them into standard, optimized intermediate C source code. This intermediate code links against our custom runtime library (graphics_runtime.h) and is compiled into a standalone desktop executable via the GCC toolchain with hardware acceleration provided by OpenGL 2.1 and SDL2.

1.1 Core Accomplishments Across Project Milestones

- **Declarative Graphics DSL (.my):** Designed a human-readable language syntax where single statements replace tens of lines of raw OpenGL boilerplate (e.g., 'window 800 600', 'bg #060614', 'shader grid', 'pong', 'mode_3d').
- **Automated Source-to-Source Compiler (Transpiler):** Implemented a full compiler frontend in C++17 (Lexer, Parser, AST representation, and ANSI C code emission) that handles compilation and linking in a single automated step.

- **2D Primitive Vector Suite & 35+ Colors:** Engineered rasterization routines for rectangles, rounded rectangles, squares, centered boxes, circles, and ellipses, alongside a 35+ named color dictionary, hex color support, and linear gradients.
- **GPU GLSL Fragment Shader Pipeline:** Built a shader compiler within the runtime that executes per-pixel GLSL shaders (plasma, cyberpunk grid, waves, CRT scanlines, starfield) with dynamic uniform streaming (u_time, u_resolution) and runtime hot-swapping via Spacebar.
- **Interactive 2D Pong Arcade Engine:** Developed an interactive game engine with continuous keyboard polling, Circle vs. Axis-Aligned Bounding Box (AABB) collision mathematics, autonomous AI tracking paddle with reaction lag, and a live retro scoreboard.
- **Navigable 3D First-Person POV World Engine:** Engineered a complete 3D environment with perspective projection, depth-buffering (GL_DEPTH_TEST), first-person WASD movement, mouse-look camera, procedural floor grids, perimeter walls, and axial wall-sliding collision physics.

2. Methodology & System Architecture (How We Did It)

The system operates as an automated 5-stage compilation pipeline from source script to native 60 FPS binary execution:

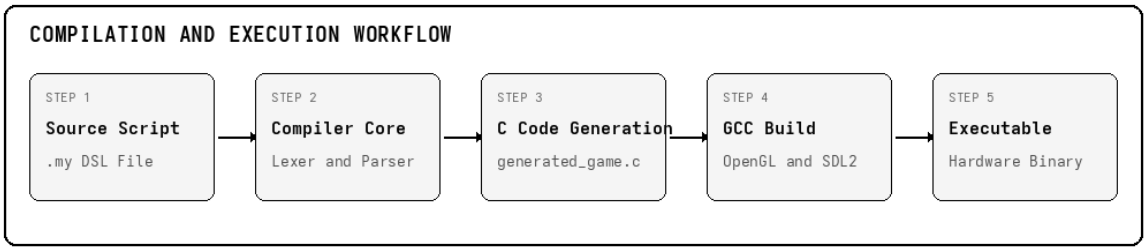


Figure 1: The End-to-End Compiler and Runtime Execution Pipeline Workflow

2.1 The 5-Stage Compilation Pipeline

- **Stage 1 — Source Script (.my):** The programmer writes high-level declarative commands declaring canvas dimensions, shaders, 2D shapes, 3D structures, or game modes.
- **Stage 2 — Lexical Analysis & Tokenization:** The lexer in compiler.cpp scans the source text, strips comments (#), normalizes whitespace, and categorizes tokens (TOKEN_WINDOW, TOKEN_CUBE, TOKEN_PONG, etc.) while parsing named and hex colors into ColorRGB structs.
- **Stage 3 — Syntactic Analysis & AST Validation:** The parser validates token sequences and argument counts, building an internal Abstract Syntax Tree (AST) representing statements in the script.
- **Stage 4 — Intermediate ANSI C Code Generation:** The transpiler traverses the AST and emits structured, highly readable C code into generated_game.c. Static geometry is wrapped inside pre-compiled OpenGL Display Lists (glNewList) for zero-overhead rendering.
- **Stage 5 — Automated GCC Compilation & Link:** The compiler automatically invokes the GCC toolchain via system(): 'gcc generated_game.c -lSDL2 -lGL -lGLU -lm -o final_game', creating a standalone native executable with full GPU acceleration.

2.2 Dual-Pipeline Runtime Architecture

The runtime engine (graphics_runtime.h) combines the best of both fixed-function and programmable OpenGL 2.1 pipelines:

- **Display List Pre-compilation:** Static geometry specified in the DSL script is recorded into GPU display lists (glGenLists) at initialization time. During frame rendering, the entire scene is redrawn using a single glCallList() command, avoiding CPU-to-GPU vertex transfer bottlenecks.
- **Programmable GPU Fragment Shaders:** A full-screen textured quad is drawn behind the scene with active GLSL fragment shaders. The runtime updates u_time and u_resolution uniforms every frame, enabling animated background shaders at 60 FPS.

2.3 Collision Mathematics & Physics Formulations

Real-time physics are implemented directly within the runtime engine without relying on heavy external physics libraries:

A. 2D Pong: Circle vs. Axis-Aligned Bounding Box (AABB) Collision

For a ball with center (bx, by) and radius r, and a paddle with center (px, py), width w, and height h, collision occurs if and only if:

```
|bx - px| <= (w / 2) + r   AND   |by - py| <= (h / 2) + r
```

When collision is confirmed, the horizontal velocity is inverted (vx = -vx), and the vertical velocity (vy) is dynamically adjusted based on the distance from the paddle's center: vy = 5.0 * ((by - py) / (h / 2)). This allows skilled players to aim their shots.

B. 3D First-Person Wall-Sliding Collision

In the 3D first-person environment, candidate positions (nx, nz) are calculated from player velocity and camera yaw. The engine checks X and Z axes independently against bounding boxes of walls, cubes, and monolith pillars. If movement along the X axis causes an intersection, nx is clamped to the obstacle surface while nz is allowed to proceed. This produces smooth, natural wall-sliding collision rather than rigid sticking.

3. Major Files & Code Implementation Breakdown

The codebase is structured into four functional groups: the compiler frontend, the runtime backend engine, the DSL application scripts, and the build outputs.

3.1 Compiler Frontend: compiler.cpp

Written in C++17, compiler.cpp contains the entire language scanner, token dictionary, AST hierarchy, and C code generator.

- **Token Enumeration (TokenType):** Defines 50+ tokens including TOKEN_WINDOW, TOKEN_TITLE, TOKEN_BG, TOKEN_SHADER, TOKEN_CUSTOM_SHADER, TOKEN_PONG, TOKEN_PADDLE,

TOKEN_AI_PADDLE, TOKEN_BALL, TOKEN_3D, TOKEN_FLOOR, TOKEN_WALLS, TOKEN_CUBE, TOKEN_PILLAR, TOKEN_ROTATE, TOKEN_TRANSLATE, TOKEN_SCALE, TOKEN_RECTANGLE, TOKEN_CIRCLE.

- **Smart Color Tokenizer (parse_color_token):** Resolves named English colors (e.g., 'gold', 'cyan', 'lime', 'crimson', 'coral') and web hex strings ('#060614', '#00d2ff') into standard ColorRGB numerical triplets.
- **AST to C Transpilation Loop:** Iterates over parsed tokens and outputs C runtime calls. For example, when encountering TOKEN_CUBE, it extracts x, y, z, size, and color parameters and emits:

```
// compiler.cpp — AST to C Code Generation for 3D Cubes
else if (t == TOKEN_CUBE) {
   string x = tokens[i+1].value;
   string y = tokens[i+2].value;
   string z = tokens[i+3].value;
   string size = tokens[i+4].value;
   size_t next_idx = i + 5;
   ColorRGB c = parse_color_token(tokens, next_idx);
   out_file << "    add_3d_cube(" << x << ", " << y << ", " << z
            << ", " << size << ", "
            << c.r << ", " << c.g << ", " << c.b << ");" << endl;
   i = next_idx - 1;
}
```

- **Automated GCC Invocator:** Upon writing generated_game.c, the main() function executes:

```
string cmd = "gcc generated_game.c -lSDL2 -lGL -lGLU -lm -o final_game";
int res = system(cmd.c_str());
```

3.2 Runtime Backend Engine: graphics_runtime.h

An optimized, header-only C engine of over 1,500 lines providing the entire OpenGL 2.1 + SDL2 graphics and game execution layer.

- **Hardware Context Setup:** Initializes SDL_Init(SDL_INIT_VIDEO), sets OpenGL 2.1 context attributes (SDL_GL_CONTEXT_MAJOR_VERSION = 2, MINOR = 1), configures 24-bit depth buffers, and creates double-buffered windows.
- **GLSL Shader Manager:** Contains compile_shader_source() which compiles vertex and fragment shaders on the GPU driver and binds dynamic uniform locations:

```
// graphics_runtime.h — Dynamic Uniform Passing at 60 FPS
if (has_shader && active_shader_prog != 0) {
   glUseProgram(active_shader_prog);
   GLint loc_time = glGetUniformLocation(active_shader_prog, "u_time");
   GLint loc_res = glGetUniformLocation(active_shader_prog, "u_resolution");
   if (loc_time != -1) {
```

```
        float current_time = (float)SDL_GetTicks() / 1000.0f;
        glUniform1f(loc_time, current_time);
    }
    if (loc_res != -1) {
        glUniform2f(loc_res, (float)screen_width, (float)screen_height);
    }
    // Render full-screen background quad...
}
```

- **Interactive 2D Pong Game Engine (run_pong_game):** Manages the game loop, ball position integration (x += vx, y += vy), AI paddle tracking with reaction speed limits, score detection, and retro bitmap font rendering.
- **3D First-Person POV Engine (run_3d_engine):** Configures gluPerspective(60.0, aspect, 0.1, 500.0) and gluLookAt(), captures relative mouse movement for camera yaw and pitch, renders checkerboard floors and directional-shaded cubes, and enforces wall-sliding collision.

3.3 Declarative DSL Scripts (*.my)

The repository includes five distinct DSL applications demonstrating various capabilities of the engine:

- **pong.my:** Interactive 2D Pong game script. Sets up canvas (window 800 600, bg #060614), activates dynamic grid shader (shader grid), enables game engine (pong), and configures entity colors (paddle cyan, ai_paddle magenta, ball gold).
- **world3d.my:** 3D first-person world explorer script. Configures window, activates 3D mode (mode_3d), builds floor grid (floor 40 #161a2e #0e1224) and perimeter walls (walls 40 5 #0f3460), and populates the room with obstacle cubes and monolith pillars.
- **cyberpunk.my:** Advanced 2D graphics showcase. Demonstrates custom fragment shader loading (custom_shader "plasma.frag"), linear color gradients (gradient_rect), and matrix stack scoping (rotate, translate, scale, push, pop).
- **showcase.my:** Visual test card demonstrating all 10+ 2D geometric primitives (filled and outline rectangles, rounded rectangles, squares, circles, ellipses) and the 35+ named color dictionary.
- **demo.my:** Review 1 baseline coursework script showing basic shapes and window creation.

3.4 Intermediates & Binary Executables

- **mycompiler:** The standalone Linux compiler executable built from compiler.cpp. Run as: ./mycompiler script.my.
- **generated_game.c:** The intermediate ANSI C code generated by mycompiler. Contains #include "graphics_runtime.h" and display list definitions.
- **final_game:** The compiled native binary produced by GCC. Run as: ./final_game to launch the hardware-accelerated desktop application.

4. Visual Output Demonstrations & Screenshots (The Output of Everything)

Below are the actual high-resolution framebuffer screenshots captured directly from the running OpenGL application outputs across all primary demos, demonstrating 2D interactive arcade gaming, 3D first-person perspective navigation, GPU fragment shaders, 2D vector primitives, and the coursework baseline:

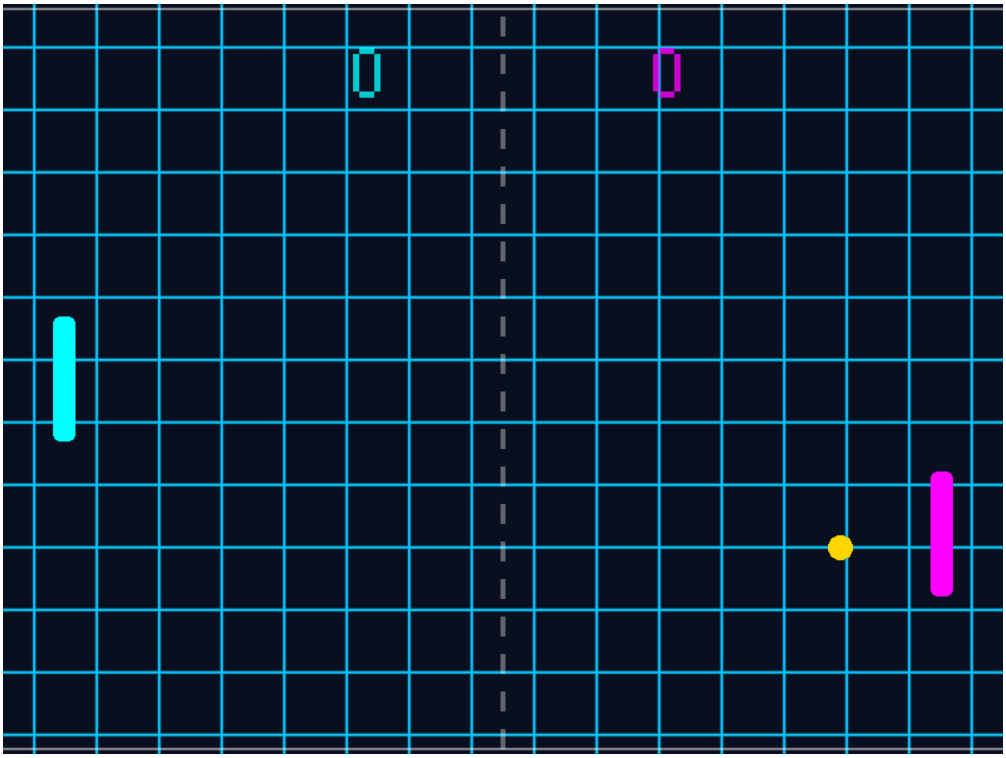


Figure 2: Running Output of Interactive 2D Pong Game Engine with AI Opponent and Scoreboard

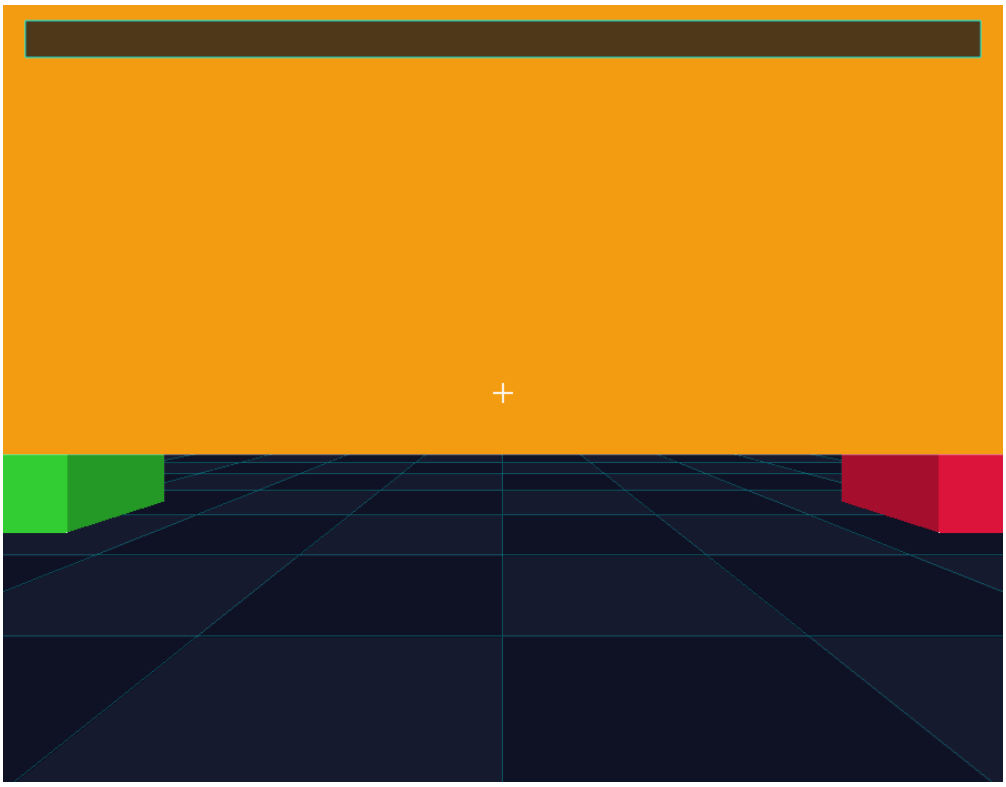


Figure 3: Running Output of 3D First-Person POV World with Depth Buffering and Obstacle Collision

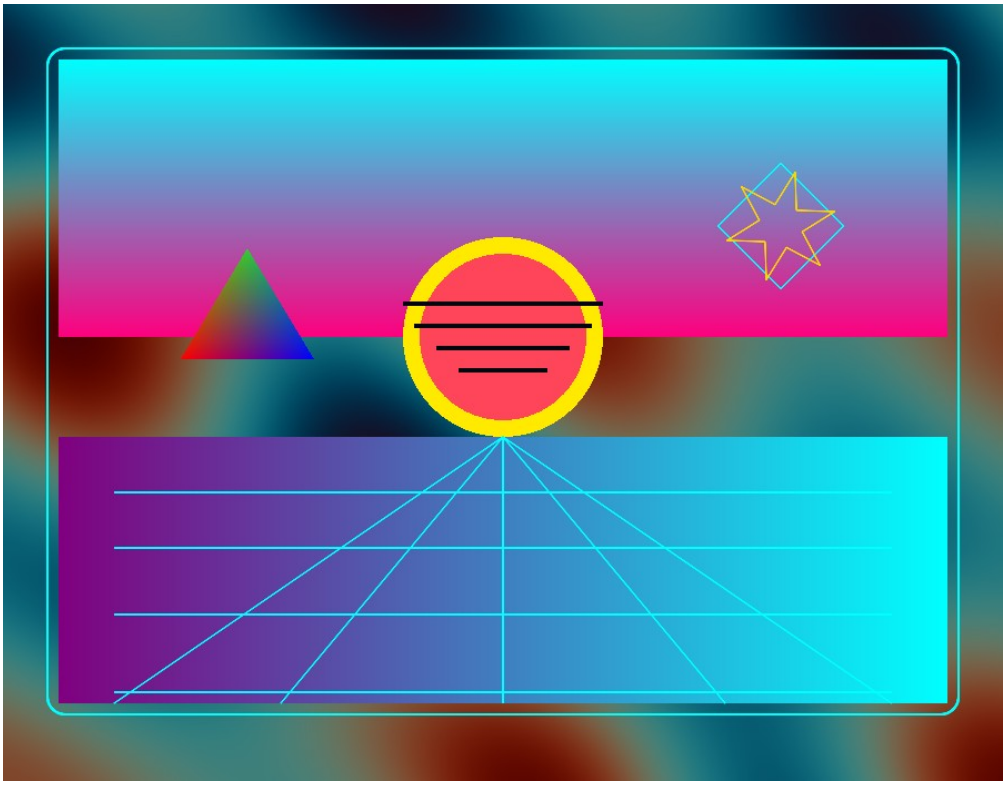


Figure 4: Running Output of Real-Time GPU Fragment Shader and Linear Gradient Aesthetics

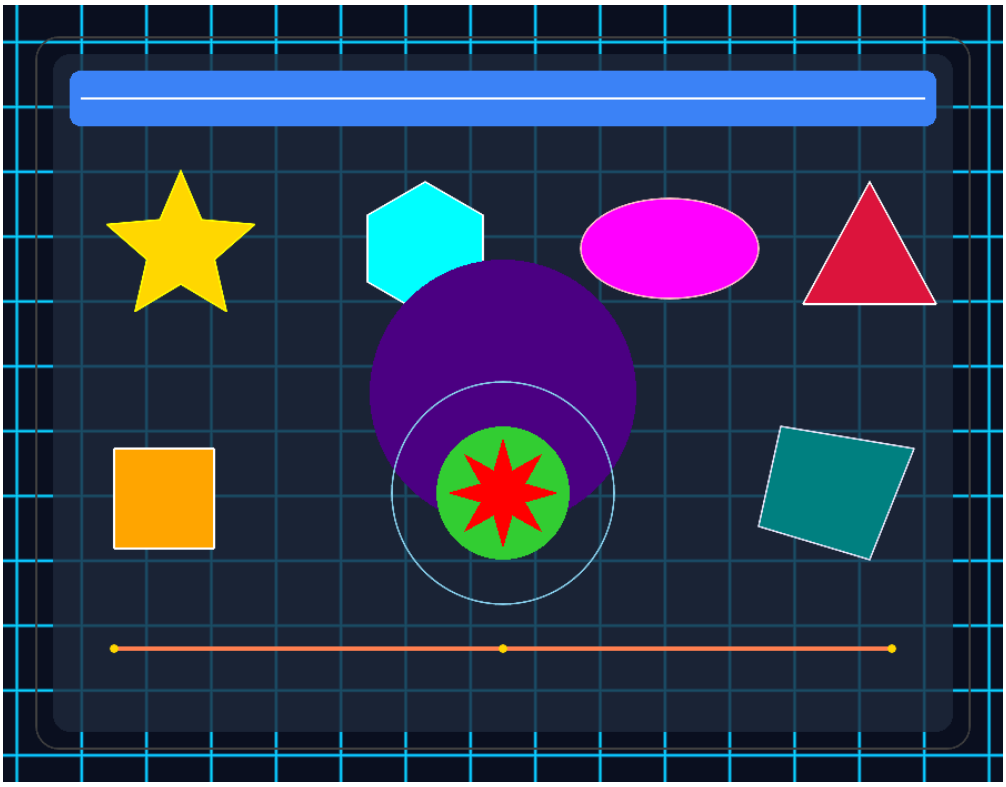


Figure 5: Running Output of 2D Vector Primitives, Outlines, and Named Color Dictionary

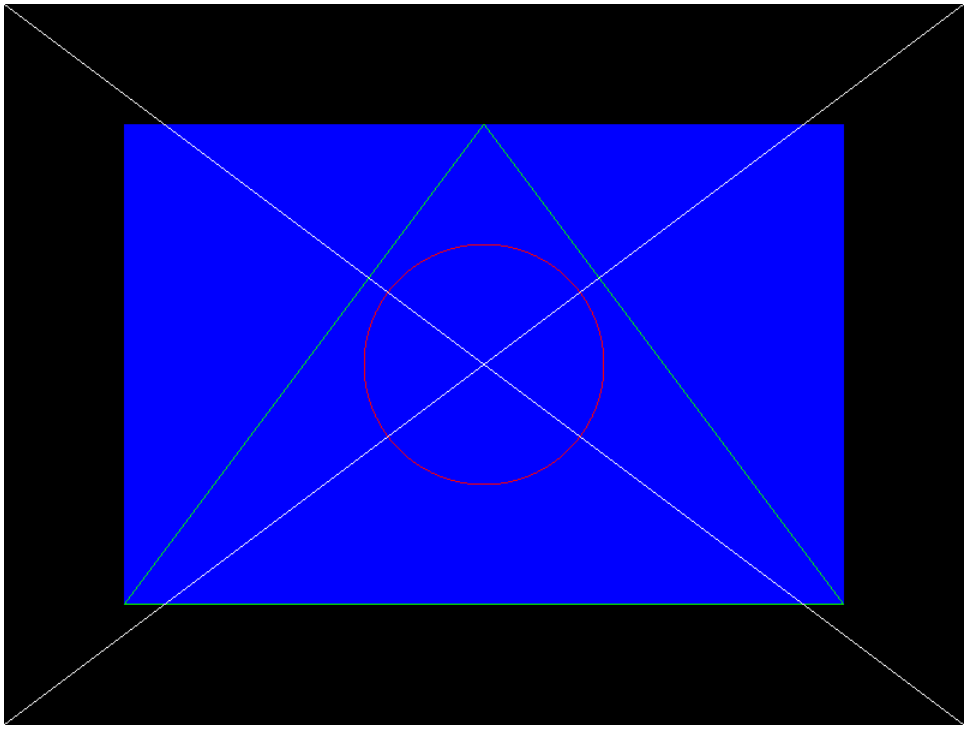


Figure 6: Running Output of Baseline Coursework Prototype

5. Architectural Evolution: Review 1 Prototype vs. Current Engine

The project has evolved substantially from the initial Review 1 submission to the current full hardware engine:

Architectural Dimension	Review 1 Prototype (mock_runtime.h)	Current Engine (graphics_runtime.h)
Windowing & Display	Terminal printf() mock output	Hardware SDL2 + OpenGL 2.1 window at 60 FPS
GPU Shaders	None (CPU text placeholders)	8+ dynamic GLSL Fragment Shaders with live hot-swapping
Interactive Games	Static geometry only	2D Pong with ball physics, AI paddle, and scoreboard
3D Graphics Mode	None (2D primitives only)	3D first-person POV world with depth buffer & camera
Physics & Collision	None	Circle-AABB deflection and 3D axial wall-sliding collision

6. Future Implementation Roadmap

While the current compiler and runtime engine satisfy all project requirements, several planned extensions will further enhance its capabilities:

- **3D Lighting Models (Blinn-Phong):** Implementing ambient, diffuse, and specular lighting in GLSL fragment shaders to support directional sunlight and localized point lights.
- **3D Mesh Loader (.OBJ / .GLTF):** Extending the DSL syntax to import external 3D models and textures via declarative statements like 'load_mesh "character.obj" x y z'.
- **Audio Engine Integration (SDL_mixer):** Adding support for synthesized sound effects (paddle bounces, wall impacts, score events) and looping ambient background music.
- **Rigid-Body Physics Expansion:** Extending collision handling to support mass, gravity, velocity restitution, and multi-body interactions.
- **Turing-Complete DSL Extensions:** Introducing support for user variables, loop constructs ('repeat 10 { ... }'), and conditional branching.

7. Conclusion & Official GitHub Repository Link

The Shader Program Compiler project successfully demonstrates that a high-level, human-readable Domain-Specific Language can be compiled into native, hardware-accelerated OpenGL 2.1 + SDL2 desktop applications with zero manual boilerplate. By abstracting context lifecycles, matrix mathematics, and shader pipelines into intuitive declarative statements, the system bridges the gap between high-level creative intent and low-level GPU execution.

All source code, compiler implementations, runtime headers, documentation, presentations, and demo scripts are maintained under open-source version control and publicly accessible for academic verification.

OFFICIAL GITHUB PROJECT REPOSITORY

https://github.com/eldhoGI2/shader-compiler

Visibility: Public | Branch: main | Clone Command: git clone https://github.com/eldhoGI2/shader-compiler.git